Supplementary Material for "PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers"

This supplement contains Appendices A–D of the main paper. Section, figure, and table numbers of the form "Section III-A" or "Table VII" refer to the main paper; numbers local to this document refer to the supplement itself.

Appendix A
The 80-Qubit Suite: Reconstruction Detail

The 80-qubit suite (Section IV-B of the main paper) was generated with `mqt.bench` 2.2.2 [1] (device-independent generation followed by transpilation to our basis, since the installed `mqt.bench`'s target-aware native-gate path raised an unrelated `qiskit` compatibility error on every circuit we tried). AE at this size is not directly producible by stock `mqt.bench`: its phase-estimation construction evaluates $\mathrm{Operator}(Q)^{2^j}$ for each evaluation bit j, and the accumulated floating-point non-unitarity trips `qiskit`'s unitarity check (`"Input matrix is not unitary"`) beyond roughly 27 evaluation qubits, which is why published AE files stop near 54 qubits. Since the Bernoulli operator Q is a single-qubit rotation, we instead supply the power analytically, $Q^{2^j} = R_Y(2^{j+1}\theta_p)$ (R_Y periodic in 4π), and otherwise run `mqt.bench`'s own native-gate transpilation unchanged. The CNOT sub-sequence—which alone determines routing—is decomposition-invariant in these rotation angles and is byte-identical to stock `mqt.bench` at every size the latter can build (verified at 10, 20, 25, and 28 qubits); only the routing-irrelevant single-qubit angles carry floating-point error at high bit indices.

QNN required an analogous reconstruction. Between `mqt.bench` 1.1.0 (which produced the deep 8,126-CX 64q QNN this suite uses) and the installed 2.2.2, the benchmark's definition was changed from a ZZ feature map (full all-pairs entanglement) plus a `RealAmplitudes` ansatz to a plain Z feature map (no entanglement) plus the same ansatz. The current generator therefore emits a near-linear 79-CX circuit at 80 qubits whose two-qubit interaction graph is *identical* to GHZ's — a degenerate duplicate, not a distinct benchmark. We restore the original deep QNN by rebuilding the 1.1.0 construction (ZZ feature map, `reps`=2, full entanglement; `RealAmplitudes`, `reps`=1) and transpiling it to our basis; at 64 qubits this reproduces the checked-in 1.1.0 file's CNOT sub-sequence byte-for-byte (identical ordered pair sequence, all $\binom{64}{2}$ interacting pairs, 8,126 CX), so the 80-qubit instance (a fully-connected $\binom{80}{2}$-edge interaction graph, 12,718 CX) is a faithful extrapolation of the same construction.

Appendix B
External Layouts from pytket-dqc

As a third, independent layout source complementing the shared-layout protocol of Section IV-C of the main paper, we route DSABRE and PORTER from a placement generated by `pytket-dqc`'s `HypergraphPartitioning` allocator [2] (best-of-3 KaHyPar seeds on a `NISQNetwork` matching our core graph), converted to a physical layout by the same topology-aware intra-core assignment DSABRE's own pytket benchmark uses [3]: each core's assigned logical qubits are placed by descending interaction-graph degree onto physical positions ordered by ascending intra-core centrality, then locally refined by a bounded pairwise-swap pass. TELESABRE is excluded for the same structural reason as in Section IV-C (no externally supplied layout support in its CLI). `HypergraphPartitioning` only decides which *core* each logical qubit goes to; the capacity it sees for a core is the length of that core's declared qubit list in the `NISQNetwork`. Following DSABRE's stock corner-removal convention (the four lowest-degree corner qubits of the whole-chip graph; Section IV-C), we withhold those four corner qubits from the cores' declared capacity and from the later physical-placement step within each core. This guarantees only a four-qubit margin *chip-wide*, not per-core, so a core can still end up fully packed; rather than skip the run in that case, PORTER's general safeguard against fully packed cores (Section III-A) legalizes the layout automatically before routing begins. DSABRE's teledata-only mechanism does not need a free port on the side it teleports *from* and was never affected by this precondition. Because `pytket-dqc` uses gate teleportation, not state teleportation, its own reported cost is in a different unit (e-bits per Steiner-tree-amortised hyperedge, not EPR pairs per qubit move) and is included only as context, following DSABRE's own established convention for this comparison [3].

Table I reports a complete grid: PORTER's full-core safeguard legalizes the three layouts that would otherwise violate its teleport precondition (25q QNN, 64q AE, 64q QNN), at the cost of the layout-quality hit an emergency eviction inevitably takes – most visibly on 64q QNN, where the evicted layout lands PORTER at 1,031 EPR pairs against DSABRE's 509, its worst relative result on this suite. Excluding that outlier, results split close to evenly: PORTER uses fewer EPR pairs on 25q Graphstate/QNN/Random and 64q AE/QFT, DSABRE on 25q AE/QFT and 64q Graphstate/Random, and the two tie on both GHZ rows; geometric-mean EPR across all twelve rows is within 1.4% of parity (DSABRE 69.2 vs. PORTER

TABLE I
PYTKET-MAPPING SHARED-LAYOUT COMPARISON (25Q AND 64Q SUITES), CORNERS-REMOVED NISQNETWORK CAPACITY. PYTKET E−BITS IS HYPERGRAPHPARTITIONING'S OWN COST IN A DIFFERENT UNIT (CONTEXT ONLY, NOT EPR-COMPARABLE). PORTER'S GENERAL FULL-CORE SAFEGUARD LEAVES NO MISSING ENTRIES ON THIS SUITE.

Suite	Circuit	pytket e-bits	DSABRE		PORTER	
			EPR	SWAP	EPR	SWAP
25q	ae	78	67	339	73	348
25q	ghz	3	3	14	3	17
25q	graphstate	4	5	24	4	19
25q	qft	70	92	437	118	438
25q	qnn	148	168	848	127	703
25q	random	564	440	1482	395	1340
64q	ae	264	361	2243	223	1638
64q	ghz	5	5	74	5	62
64q	graphstate	10	8	68	10	60
64q	qft	307	320	2112	278	2021
64q	qnn	621	509	8215	1031	6538
64q	random	1020	750	3969	861	4384

70.2). Pooled with the TELESABRE-layout grid of Table VII of the main paper, PORTER uses fewer EPR pairs on fifteen of the thirty fixed-layout instances, DSABRE on twelve, with three ties (Section IV-E).

The grid above feeds pytket-dqc's partition to PORTER as a controlled probe. The complementary pipeline question – offering the same partition to PORTER as an optional extra layout candidate beside its own, which lowers geometric-mean EPR by a further 16% on the most partitionable circuits but is kept opt-in on cost and robustness grounds – is a configuration study of PORTER itself rather than part of this comparison, and is deferred to Appendix C.

APPENDIX C
PYTKET-DQC AS AN OPT-IN PORTER LAYOUT CANDIDATE

Appendix B feeds pytket-dqc's partition to PORTER as a controlled probe; the same partition can also be offered to PORTER as one extra layout candidate beside its own (Section III-A of the main paper). Since PORTER routes far faster than pytket-dqc partitions, this is nearly free on the routing side; taking the better of the two per instance lowers the eighteen-instance geometric-mean EPR from 116.8 to 98.6 (-16%), with the gains concentrated on the most partitionable circuits (Graphstate $7\to4$, $20\to10$, $28\to11$ across the three suites; 80q AE $490\to352$). We keep it *opt-in* rather than the default for three reasons. First, on the dense and Random circuits the external partition is often *worse* for PORTER (64q QNN $864\to1{,}031$, 64q Random $737\to861$), so the best-of simply retains PORTER's own layout there. Second, the partition itself is expensive – seconds to minutes, and on dense circuits more than PORTER's entire routing pass (309 s to partition 80q Random versus $\approx$37 s to route it; 16 s versus 0.6 s on 80q AE) – against a sub-millisecond cost for PORTER's own layout build, so adopting it by default would erase PORTER's compile-time advantage (Section IV-G of the main paper). Third, it reintroduces the external-partitioner dependency PORTER's self-contained layout avoids, and unlike it can leave a core fully packed, relying on the full-core repair of Section III-A. PORTER therefore defaults to its own layout and treats an external partition as an optional refinement, selected only when EPR is at a premium and its generation time is acceptable.

APPENDIX D
INDEPENDENT REPLAY VERIFICATION

Beyond code review, we built an independent replay verifier that treats the router as a black box: it takes only the emitted `CompiledOp` stream, the original circuit, and the initial layout, and re-derives, from scratch, whether the stream is a faithful and physically legal execution. The verifier maintains its own physical-position map and a per-logical-qubit FIFO queue of pending gate IDs (derived directly from the circuit, independent of any router-internal DAG state), and checks, per emitted operation:

- every gate is popped from the head of *every* one of its operand qubits' queues before being marked executed, so gates cannot execute out of DAG order or twice;
- every `local_swap`/`local_cx` acts on a genuine intra-core coupling edge, at the physical positions the verifier's own simulated layout says the operand logical qubits occupy;
- every `teleport` uses the correct inter-core port pair for its two cores, both ports were free beforehand, and the source qubit was coupling-adjacent to the source port;
- every `cat_entangler`, `cat_cx`, and `cat_disentangler` triple observes port-adjacency and free-port preconditions, and no other operation touches either communication port while a cat section is open;
- the emitted `RouterStats` counters (EPR, teleport, telegate, packet, SWAP, local-CX, single counts) match the counts the verifier itself derives from the op stream, catching any bookkeeping drift between what is emitted and what is counted.

We ran the verifier against 60 randomized circuits (4–40 logical qubits, 20–600 gates, both architectures, seeds 0–59) and against every circuit in the evaluation suite (Section IV of the main paper); all pass. Two further artefacts make these checks concrete: a *certified trace* (Figure 1) and a *fault-injection study* (Table II).

A certified trace. Figure 1 replays the stream PORTER emits for the running example of the main paper (Section III-G) — fourteen operations exercising every op type — and lists, per operation, exactly what the verifier asserts before accepting it. The trace makes the three primitives' legality conditions operational: the TELEGATEPACKET is one *bracketed section* in which the two reserved ports are untouchable and every interleaved SWAP must carry packet provenance; the TELEDATA at op 11 must name the unique port pair of its core pair, find both ports free, start port-adjacent, and land exactly on the destination port; and every gate execution, local or remote, is popped from the head of each operand's pending-gate queue, so DAG order is enforced gate by gate rather than assumed from the router's own bookkeeping.

#	emitted operation		verifier asserts
0	`swap` $p_3 \leftrightarrow p_4$		coupling edge; operands sit where the replayed layout says
1	`cat_entangler` q_0; (p_5, p_{12}); 1 EPR		no section already open; *exact* port pair of (c_0, c_1); both ports free; q_0 adjacent to p_5; opens section, reserving p_5, p_{12}
2	`swap` $p_{10} \leftrightarrow p_9$	route q_1	tagged as packet routing; touches neither reserved port
3	`cat_cx` g_1 (p_{12}, p_9)		g_1 at the head of *both* q_0's and q_1's queues; control = section control; proxy is the target-side port p_{12}; target in c_1, adjacent to p_{12}
4	`swap` $p_{13} \leftrightarrow p_{14}$	route q_2	as op 2
5	`cat_cx` g_2 (p_{12}, p_{13})		as op 3
6	`cat_cx` g_3 (p_{12}, p_{15})		as op 3 (q_3 already port-adjacent)
7	`cat_disentangler`		a section is open; closes it, releasing the ports
8	`swap` $p_6 \leftrightarrow p_7$		coupling edge
9	`local_cx` g_5 (p_4, p_7)		queue heads; same core; coupling edge
10	`swap` $p_{13} \leftrightarrow p_{14}$		coupling edge
11	`teleport` q_5: $p_{13} \rightarrow p_5$; 1 EPR		cores adjacent; exact port pair (p_{12}, p_5); both ports free; q_5 adjacent to p_{12}; lands exactly on p_5
12	`swap` $p_4 \leftrightarrow p_5$		coupling edge
13	`local_cx` g_4 (p_3, p_4)		queue heads; coupling edge

Fig. 1. The full operation stream PORTER emits for the running example of the main paper (Section III-G, Figure 5), as replayed by the verifier. Ops 1–7 execute inside the live cat section, where only packet-tagged SWAPs and 1Q gates that avoid the two reserved ports are legal. At end of stream the verifier further asserts: no cat section left open, all five gates executed exactly once, and the emitted statistics equal to the derived counts (2 EPR, 1 teleport, 1 packet, 0 single telegates, 6 SWAPs, 2 local CXs).

TABLE II
FAULT-INJECTION VALIDATION OF THE VERIFIER (`VERIFY_FAULT_INJECTION.PY`): SINGLE-POINT CORRUPTIONS APPLIED TO OPERATION STREAMS THAT PASS VERIFICATION, EACH OF WHICH MUST BE REJECTED.

Injected corruption	Rejecting check
delete a `cat_disentangler`	section state machine
swap a teleport's two ports	exact inter-core port pair
zero a teleport's EPR cost	per-op EPR accounting
duplicate a gate-carrying op	every gate exactly once
delete a gate-carrying op	queue order / end-of-stream coverage
swap two ops sharing a qubit	per-qubit dependency order
retarget a SWAP to a non-coupled pair	coupling-edge legality
untag an in-section routing SWAP	cat-section isolation
retarget an in-section SWAP onto a port	reserved-port protection
increment `stats.epr` by one	stats–stream cross-check

Fault injection. On streams that always pass, a verifier that checks nothing is indistinguishable from one that checks everything, so we also certify that each check *fires*: `verify_fault_injection.py` takes passing op streams and applies the ten single-point corruptions of Table II, one mutant per corruption, spanning misuse of all three communication primitives (wrong ports, free-port violations, broken cat-section bracketing), gate-coverage and dependency-order violations, illegal SWAPs, and statistics drift. Across the running example (which admits all ten corruption classes) and thirty fuzzed circuits on both architectures, all 276 applicable mutants are rejected, each by the intended check; the same harness runs on any benchmark QASM file.

REFERENCES

[1] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[2] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[3] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.